Vega: A Keystroke-Accurate Scientific Calculator for Android

Emulating the Casio fx-82MS as a Finite State Machine in Kotlin and Jetpack Compose

A. Simoes
americosimoes@zenith-node-alpha
Independent Project

October 2026

Abstract

We present **Vega**, an Android scientific calculator whose sole design goal is behavioural fidelity to the classic Casio fx-82MS hardware. Rather than adopting the prevailing expression-parser architecture of modern calculator apps, Vega reproduces the device's keystroke-driven finite state machine: a two-operand register model in which every key press mutates a small, explicitly enumerated state, and in which there is no intermediate symbolic expression. We describe the state machine, the operator-precedence semantics it produces (deliberately not infix precedence), the Compose-based UI that mirrors the 5×8 physical key matrix, and the modern build toolchain (AGP 9.0, Kotlin 2.2, Compose BOM 2025.12) used to ship it. We argue that for "retro emulation" software, keystroke semantics—not algebraic convenience—are the correct contract.

Contents

1 Introduction **2**

2 Related Work and Design Space **2**

3 System Architecture **2**
3.1 Build stack . . . 2
3.2 Module layout . . . 3

4 The Keystroke State Machine **3**
4.1 Registers . . . 3
4.2 The dispatcher . . . 4
4.3 Binary core and error semantics . . . 5
4.4 Unary functions . . . 5

5 Deliberate Non-Precedence **5**

6 User Interface **6**
6.1 Layout fidelity . . . 6
6.2 Two-line LCD . . . 6
6.3 Compose structure . . . 6

7 Deliberate Limitations **6**

8 Correctness Argument **7**

9 Conclusion 7

1 Introduction

Modern mobile scientific calculators overwhelmingly adopt one of two architectures: (i) a full expression parser that accepts infix text (e.g. `2+3*4`) and evaluates it with standard operator precedence, or (ii) a symbolic engine that supports variables, CAS operations, and history. Both are excellent *tools*, but neither is a faithful *emulation* of the hardware scientific calculator as it actually behaves.

A Casio fx-82MS, by contrast, operates on a strict *keystroke register machine*:

- It never sees a complete expression. It has *one* pending operand, *one* pending operator, and *one* entry.
- Consequently, `2 + 3` $\times 4 = yields$ `20`$, not 14. This is not a bug; it is the documented behaviour of the physical devic$ $applies the last binary operation using the last entered right operand (Casio's "constant" mode).$
- `AC` resets everything; `C` (here `DEL`) clears only the current entry.
- `SHIFT` and `ALPHA` latch for exactly one subsequent keystroke.
- Unary functions (`sin`, $\sqrt{\ }$, x^2, x^{-1}) apply *immediately* to the current entry and set a "fresh entry" flag, so the next digit *replaces* rather than appends.

These behaviours are the contract Vega honours. The contribution of this paper is therefore not a novel algorithm but a *design argument*: for emulation software, the state machine is the specification, and no parser shortcut is acceptable.

2 Related Work and Design Space

Three families of mobile calculator exist.

1. **Parsers.** Accept free-form infix (e.g. *Calculator++*, *HiPER Scientific*). Convenient but produce `2+3` $\times$ `4` $= 14$, contradicting hardware behaviour.
2. **Keypad front-ends to a parser.** Visually mimic a Casio but secretly tokenise the display into an expression and call a CAS at `=`. The *look* is faithful; the *feel* is not, because corner cases (`=` repetition, unary-then-digit, chained operators) diverge.
3. **State machines.** Genuine emulation of the register model. Rare in the modern app ecosystem because it is "harder to get right" and offers no marketing hook.

Vega is in category 3. The remainder of the paper details its implementation.

3 System Architecture

3.1 Build stack

Vega targets the modern Android toolchain as of October 2026:

Component	Version / note
Android Gradle Plugin	9.0.1 (built-in Kotlin support)
Kotlin	2.2.10 (embedded in AGP 9.0.1)
Compose Compiler plugin	`org.jetbrains.kotlin.plugin.compose` 2.2.10
Compose BOM	2025.12.00
`activity-compose`	1.11.0
`compileSdk` / `targetSdk`	36
`minSdk`	24
JDK	17

AGP 9.0 embeds the Kotlin compiler directly, which removes the need for the standalone `org.jetbrains.kotlin.android` plugin. Compose, however, still requires the *separate* `org.jetbrains.kotlin` plugin (Kotlin 2.0+ requirement), whose version must match the AGP-embedded Kotlin version exactly. A mismatch produces the compiler-version error seen during early development.

3.2 Module layout

The app is a single Android module, `:app`, with:

- `MainActivity.kt` — `ComponentActivity` hosting the Compose tree.
- `CalcState.kt` (conceptually in the same file) — the immutable data class holding all registers.
- `press(state, key)` — the pure keystroke dispatcher.
- `calc(a,b,op)` — the two-operand arithmetic core.
- `formatNum(v)` — display formatter with automatic scientific notation.

The state is fully immutable; every keystroke returns a new `CalcState`. Compose observes it via `mutableStateOf`, so there is no mutable field anywhere in the business logic. This makes the calculator trivially testable with property-based tests: for any sequence of keys, the resulting state is a pure function of the input sequence.

4 The Keystroke State Machine

4.1 Registers

The machine's registers, in the terminology of the physical device, are:

Register	Type	Meaning
`entry`	`String`	The number currently being typed (display line 1).
`accumulator`	`Double?`	Left-hand side of a pending binary operation.
`pendingOp`	`String?`	One of `+`, `-`, `*`, `/`, `^`.
`lastOp`	`String?`	Last binary op, for `=`-repetition.
`lastOperand`	`Double`	Right operand of the last binary op.
`ans`	`Double`	Last computed result, for `Ans`.
`degMode`	`Boolean`	DEG vs RAD (displayed as `D`/`R`).
`freshEntry`	`Boolean`	Next digit *replaces* the display.
`errorState`	`Boolean`	Sticky "Math ERROR" until `AC`.

Key invariant. `freshEntry` is the single most important flag. It is set to `true` after: a completed `=`, a binary operator press, `AC`, or any unary-function application. It is set to `false` by: any digit, the decimal point, `Ans`, or the $\times 10^x$ key. This is exactly the hardware's behaviour.

4.2 The dispatcher

The dispatcher is a pure function; it is the entire calculator.

```
private fun press(s0: CalcState, key: String): CalcState {
    var s = s0
    if (s.errorState && key != "AC" && key != "ON") {
        s = s.copy(entry = "0", errorState = false, freshEntry = true,
                accumulator = null, pendingOp = null)
    }
    return when {
        key.length == 1 && key[0].isDigit() -> {
            if (s.freshEntry) s.copy(entry = key, freshEntry = false)
            else               s.copy(entry = s.entry + key)
        }
        key in listOf("+","-","*","/","^") -> {
            if (s.pendingOp != null && s.freshEntry) {
                s = applyPending(s)          // chain: fold accumulator
            }
            s.copy(accumulator = s.entry.toDoubleOrNull() ?: 0.0,
                   pendingOp = key, freshEntry = true)
        }
        key == "=" -> when {
            s.pendingOp != null -> {         // normal evaluation
                val b = s.entry.toDoubleOrNull() ?: 0.0
                val r = calc(s.accumulator ?: 0.0, b, s.pendingOp!!)
                if (r == null) s.copy(entry = "Math ERROR",
                                      errorState = true, freshEntry = true)
                else s.copy(entry = formatNum(r), ans = r,
                            lastOp = s.pendingOp, lastOperand = b,
                            pendingOp = null, accumulator = null,
                            freshEntry = true)
            }
            s.lastOp != null -> {             // "=" repetition
                val r = calc(s.entry.toDoubleOrNull() ?: 0.0,
                             s.lastOperand, s.lastOp!!)
                if (r == null) s.copy(entry = "Math ERROR",
                                      errorState = true, freshEntry = true)
                else s.copy(entry = formatNum(r), ans = r, freshEntry = true)
            }
            else -> s.copy(ans = s.entry.toDoubleOrNull() ?: 0.0,
                           freshEntry = true)
        }
        key == "AC" || key == "ON" -> CalcState()   // all registers to default
        key == "DEL" -> if (s.freshEntry) s
                        else if (s.entry.length <= 1) s.copy(entry = "0")
                        else s.copy(entry = s.entry.dropLast(1))
        key == "(-)" -> if (s.entry.startsWith("-"))
                            s.copy(entry = s.entry.substring(1))
                        else s.copy(entry = "-" + s.entry)
        else -> s    // unary fns, modes, cosmetic keys: see full source
    }
}
```

Listing 1: Simplified keystroke dispatcher (arithmetic and binary ops).

4.3 Binary core and error semantics

The arithmetic core is trivial; the error semantics are not:

```
private fun calc(a: Double, b: Double, op: String): Double? = when (op) {
    "+" -> a + b
    "-" -> a - b
    "*" -> a * b
    "/" -> if (b == 0.0) null else a / b        // division-by-zero
    "^" -> a.pow(b)
    else -> null
}
```

Listing 2: Two-operand arithmetic core.

A `null` return maps to `Math ERROR` and sets `errorState`, which is sticky until `AC`. This reproduces the hardware's behaviour of refusing further arithmetic until explicitly cleared.

4.4 Unary functions

Unary functions apply immediately and are the reason `freshEntry` exists:

```
key == "sqrt" -> applyUnary(s) { if (it < 0) null else sqrt(it) }
key == "x2"   -> applyUnary(s) { it * it }
key == "x3"   -> applyUnary(s) { it * it * it }
key == "inv"  -> applyUnary(s) { if (it == 0.0) null else 1.0 / it }
key == "sin"  -> applyUnary(s) {
    sin(if (s.degMode) Math.toRadians(it) else it)
}
// ... applyUnary sets freshEntry = true on success.
```

Listing 3: Unary function application (excerpt).

Because `freshEntry` becomes `true`, pressing `sin` then `5` computes sin(5) on the value *5*, not on the result of sin of anything. This is the exact behaviour of the physical key.

5 Deliberate Non-Precedence

We stress one design choice: **Vega does not implement infix operator precedence**. The input sequence

`2 + 3` $\times 4 =$
produces **20**, because the register machine folds at each operator press:

1. `2`: entry = 2.
2. `+`: accumulator = 2, pendingOp = '+'.
3. `3`: entry = 3.
4. $\times$: pendingOp exists and freshEntry is true, so *fold*: entry = 2 + 3 = 5, accumulator = 5, pendingOp = '*'.
5. `4`: entry = 4.
6. `=`: entry = $5 \times 4 = 20$.

A reviewer accustomed to algebraic calculators may read this as a bug. It is not; it is the documented Casio behaviour, and it is the entire point of the project. Vega's contribution is to make this contract explicit and testable, rather than hidden inside a parser that attempts to look like a Casio but does not feel like one.

6 User Interface

6.1 Layout fidelity

The Compose layout is a five-column by eight-row grid that matches the fx-82MS key matrix row-for-row:

SHIFT	ALPHA	MODE	ON	DEL
x^{-1}	nCr	Pol	x^3	$+$
a b/c	$\sqrt{\ }$	x^2	$\wedge$	$-$
log	ln	$(-)$	deg$'''$	$\times$
sin	cos	tan	RCL	$\div$
7	8	9	AC	$\times 10^x$
4	5	6	$=$	Ans
1	2	3	0	.

Keys are coloured by semantic class: **numeric** (`#3C3C3C`), **function** (`#555555`), **operator** (`#E58C2F`), **AC/ON** (`#B04040`), and text `#FFFFFF`.

6.2 Two-line LCD

Line 1 (top) shows the DEG/RAD indicator and, when a binary operation is pending, the accumulator and operator symbol. Line 2 (bottom, monospaced, right-aligned) shows the current entry. The background is the pale LCD green `#C8D6A2` with text ink `#161C10`, reproducing the optical character of the original LCD.

6.3 Compose structure

The UI is a single composable, `CasioCalcApp()`, holding the entire `CalcState` in a `remember { mutableStateOf(CalcState()) }`. Every button's `onClick` does exactly one thing:

```
state = press(state, label)
```

There is no other mutable state in the UI. This ensures the calculator's behaviour is (i) identical for physical button presses and programmatic tests, and (ii) recoverable after a Compose recomposition, because state is a data class rather than a scatter of mutable fields.

7 Deliberate Limitations

For scope control in this version, three things are *not* emulated:

- **Shift-latched secondary functions.** `SHIFT` and `ALPHA` latch (the top line shows `S` or `A`) but currently do nothing beyond the DEG/RAD toggle. Adding a second-layer function map is a straightforward extension: add a `shiftArmed` flag to `CalcState` and branch inside `press`.
- **Numeric systems (base-n display, HEX/OCT/BIN).** The keys are present for layout fidelity but are inert.
- **STAT / EQN / MATRIX modes.** Out of scope; the project is a COMP-mode emulator.

None of these omissions affects the keystroke semantics for the functions that *are* implemented, which is the property we care about.

8 Correctness Argument

Because `press` is pure and total, the calculator's correctness can be specified as a small set of refactorings:

1. **Determinism:** `press`(s, k) depends only on s and k.
2. **Freshness:** `freshEntry` = `true` $\Rightarrow$ the next digit key replaces `entry`.
3. **Folding:** for any sequence $d_1 o_1 d_2 o_2 =$, the result equals $\text{fold}(o_1(d_1, d_2), o_2, d_3)$ where fold is left-associative two-operand application.
4. **Stickiness:** once `errorState` is set, `press` returns the identity on every key except `AC` and `ON`.
5. **Constants:** after a completed binary op, `=` re-applies the last operator with the last right operand.

These five invariants can be expressed as QuickCheck-style properties against the pure `press`; we leave the full test suite to future work.

9 Conclusion

Vega demonstrates that the correct specification for a retro-emulation calculator is not an expression language but a keystroke state machine. The design is small (one dispatcher, one data class, one arithmetic core), the semantics are faithful, and the code is trivially testable because every operation is a pure function of a small immutable state. The build toolchain is deliberately current (AGP 9.0, Kotlin 2.2, Compose BOM 2025.12) so that the app can ship on today's Android.

The naming, **Vega**, is a deliberate choice: Vega is the reference star of the magnitude system—the point from which brightness is measured. That is the role a faithful calculator should play.

Acknowledgements

The author thanks the Android and Kotlin teams for AGP 9.0's embedded Kotlin, and the Compose team for the still-required `org.jetbrains.kotlin.plugin.compose`. The Casio fx-82MS is a trademark of Casio Computer Co., Ltd.; Vega is an independent emulation and is not affiliated with or endorsed by Casio.

References

[1] Android Developers. *Android Gradle plugin release notes: AGP 9.0*. https://developer.android.com/build/releases/gradle-plugin, 2026.

[2] Android Developers. *Set up the Compose Compiler Gradle plugin*. https://d.android.com/r/studio-ui/compose-compiler, 2026.

[3] Android Developers. *Compose BOM mapping*. https://developer.android.com/develop/ui/compose/bom/bom-mapping, 2026.

[4] Casio Computer Co., Ltd. *fx-82MS User's Guide*. 2001.

[5] Android Developers. *Jetpack Compose documentation*. https://developer.android.com/develop/ui/compose, 2026.